ДЛЯ СТУДЕНТОВ · 90 МИНУТ

Практика 1. Локальный выбор и координация

Распределение задач без LLM. Начало первой обязательной лабораторной.

Цель: реализовать две стратегии распределения задач и проверить, как правила взаимодействия влияют на допустимость и качество общего результата.

1. Постановка задачи

Есть **n исполнителей** и **n задач**. В матрице C строка i соответствует агенту Ai, столбец j — задаче Tj. Число C[i,j] — известная положительная стоимость выполнения задачи этим агентом.

Назначьте каждому агенту ровно одну задачу и каждую задачу ровно одному агенту. Минимизируйте **суммарную стоимость**. Нельзя пропускать задачи, назначать задачу дважды или давать агенту две задачи.

Стоимость	T0	T1	T2
A0	1	2	9
A1	2	15	2
A2	9	14	1

Это матрица **demo_17_vs_5** из data/cases.json. Начните с ручного разбора: что выберет каждый агент независимо и будут ли соблюдены ограничения?

Предположения этой практики

Все стоимости известны своим исполнителям и не меняются. Агенты кооперативны и сообщают правдивые цены. Сообщения доставляются без потерь; раунды последовательны. Время реальной работы, сеть, дедлайны и стратегический обман не моделируются.

Формат результата: assignment[i] = индекс задачи агента i. Индексы начинаются с нуля. Допустимый assignment — перестановка чисел от 0 до n−1. Например, [2,0,1] означает A0→T2, A1→T0, A2→T1.

Стоимость — абстрактная цена. Не путайте минимум суммы стоимостей с минимумом времени завершения всех работ.

АЛГОРИТМЫ

2. Четыре стратегии

Две реализации уже даны; две нужно дописать в starter.py.

Готово: независимый выбор и точный эталон

independent: каждый агент берёт самый дешёвый элемент своей строки. При равенстве цен выбирает задачу с меньшим индексом. Конфликты не исправляются: это намеренно слабый baseline.

exact: перебирает все допустимые назначения и выбирает минимум. При равенстве выбирает лексикографически первое назначение. Реализация в core.py предназначена только для n ≤ 8; в наборе n ≤ 7.

TODO 1: глобальный жадный алгоритм

Соберите все тройки **(стоимость, agent_id, task_id)** и отсортируйте их по возрастанию. Идите по списку: берите пару только тогда, когда и агент, и задача ещё свободны. Верните Outcome(assignment, communication_modelled=False). Не заменяйте это правилом «обходить задачи по очереди»: это другой алгоритм.

TODO 2: последовательный Contract Net

Менеджер обходит задачи в task_order. На каждом раунде запрашивает заявки у всех свободных агентов. Победитель — минимум **(цена заявки, agent_id)**. Победитель становится занятым и больше не участвует. Менеджер вызывает propose/accept и не читает приватную строку _costs.

Фаза	Кто кому и что посылает	Сколько
CFP	Менеджер объявляет текущую задачу каждому свободному агенту.	m
PROPOSE	Каждый свободный агент возвращает свою цену.	m
ACCEPT / REJECT	Победитель получает ACCEPT, остальные — REJECT.	m
INFORM	Победитель сообщает об условном завершении.	1

Квота агента — одна задача; INFORM не возвращает его в пул. В каждой фазе обрабатывайте агентов по ID. Формат сообщения: type, sender, receiver, task; для PROPOSE также bid. Например: `{"type":"PROPOSE","sender":"A0","receiver":"manager","task":0,"bid":1}`.

Это учебное упрощение Contract Net [1], а не полная реализация CNET или FIPA. Одна отправка одному адресату = одно сообщение.

ЗАПУСК

3. Файлы и команды

Python 3.10+; только стандартная библиотека. Интернет и API-ключи не требуются.

Файл	Для чего нужен
starter.py	Ваши две реализации; independent уже готов.
core.py	Типы, Contractor, проверка назначения и exact.
run.py	Запуск эксперимента, CSV, сводка, трассы JSON.
data/cases.json	30 фиксированных матриц; исходные данные не менять.
tests/	Проверки готового кода и ваших реализаций.
report_template.md	Заготовка отчёта.

Сначала — рабочий запуск готовых методов

Распакуйте студенческий ZIP. Откройте терминал в папке, где лежат run.py и starter.py. На Windows команда может называться py вместо python.

```
python -m unittest discover -s tests -v
python run.py --case demo_17_vs_5 --methods independent exact
```

До реализации два TODO дают **5 пропущенных проверок (SKIP)**. Остальные 6 проверок должны пройти. SKIP не означает, что лабораторная сдана.

После реализации — все методы и весь набор

```
python -m unittest discover -s tests -v
python run.py --out results_all
```

Ожидаются 11 успешных тестов без SKIP. В results_all появятся results.csv, summary.csv, results.json и traces.json. CSV сохранён в UTF-8 с BOM для удобного открытия в табличном редакторе.

Отдельный порядок задач

```
python run.py --case demo_17_vs_5 --methods contract_net \
  --order 1,0,2 --out results_order_102
```

Последнюю команду можно ввести одной строкой без обратного слеша. Меняйте --out, чтобы не перезаписать предыдущий эксперимент.

Автотесты проверяют реализацию, но не заменяют анализ: в отчёте нужно объяснить причины результатов.

ЭКСПЕРИМЕНТ

4. Что измерить и проверить

Не выбирайте только удачные случаи и не смешивайте несовместимые метрики.

Обязательный эксперимент

А. На demo_17_vs_5 получите назначения всех четырёх методов. Для каждого отдельно проверьте ограничения; разберите стоимость и цепочку решений.

Б. Запустите четыре метода на всех **30 матрицах**. Набор включает 6 ручных примеров и 24 сгенерированные матрицы: случайные, с дешёвой диагональю и с общей дешёвой задачей. Это учебный набор, не репрезентативный benchmark.

В. На demo_17_vs_5 переберите все **6 порядков задач** для Contract Net. Сохраните порядок, назначение, стоимость и число сообщений. Не ищите по этому примеру «универсально лучший» порядок.

Г. Разберите один раунд трассы. Покажите объявления, заявки, выбор победителя, смену состояния и условное завершение.

Метрика	Точная интерпретация
feasible	Выполнены все ограничения назначения. Сначала оцениваем эту метрику.
cost	Сумма цен допустимого назначения. Для недопустимого результата — пусто.
proposal_cost	Сумма индивидуальных предложений. Не заменяет cost при конфликтах.
gap_pct	100 × (cost − optimal_cost) / optimal_cost. Только для допустимых назначений.
conflicts / coverage	Число лишних назначений на уже выбранные задачи / доля уникально выбранных задач.
messages	Число отправок одному адресату. У центральных baseline — пусто: связь не моделируется.
elapsed_ms	Время локального Python. Не сеть, не время выполнения задач и не latency реальной MAS.

Средний gap считается только по допустимым случаям. Всегда указывайте их число: низкий gap на малом подмножестве не означает хороший алгоритм на всём наборе.

Минимальная сводная таблица

Метод | допустимые / всего | средний и максимальный gap | число оптимальных решений | среднее число сообщений. Для независимого выбора отдельно укажите долю конфликтов.

СДАЧА МАТЕРИАЛОВ · БЕЗ ЗАЩИТЫ

5. Результат практики

Можно работать в паре. Сдаём промпты, код и отчёт; отдельной защиты нет.

Время	Этап
0–10 мин	Ручной разбор и гипотеза.
10–20 мин	Запуск и чтение интерфейсов.
20–40 мин	Глобальный greedy.
40–65 мин	Contract Net и трасса сообщений.
65–80 мин	30 матриц; все 6 порядков контрпримера.
80–90 мин	Сохранить результаты и оформить выводы.

Что сдавать

Промпты: файл prompts.md с существенными запросами, моделями и инструментами. Если ИИ не использовали, явно напишите это. Внутренние скрытые рассуждения модели не требуются.

Код: starter.py с двумя реализациями; команды воспроизведения и среда. **Результаты:** results.csv, summary.csv, одна трасса и таблица всех шести порядков задач.

Отчёт: 1–2 страницы по report_template.md. Обязательны постановка и гипотеза, **результаты, анализ и выводы**; укажите ограничения и вклад участников. Таблицы и трассы можно приложить отдельно.

Критерии первой лабораторной · 10 баллов

Баллы	За что
3	Корректные greedy и Contract Net: ограничения, ничьи, состояние, сообщения.
2	Полный эксперимент: 30 матриц и все 6 порядков контрпримера.
3	Корректные метрики, результаты, анализ причин и обоснованные выводы.
2	Воспроизводимость и полнота комплекта: промпты, код, отчёт.

Вопросы для анализа в отчёте

Почему независимый выбор может показать сумму ниже оптимума? Что гарантирует протокол и чего не гарантирует? Почему порядок задач влияет на результат? Каковы ограничения симуляции?

Необязательное развитие: порядок объявлений, обмен задачами или отказ. Сравните с исходной версией. Дополнение не заменяет основной эксперимент.

[1] R. G. Smith. The Contract Net Protocol. IEEE Transactions on Computers, 1980. Учебная версия, матрицы и код — материалы курса.